

Java Full-Stack & Core Interview Guide

A comprehensive, high-yield technical reference covering Core Java, Collections, Multithreading, Modern Java (8–21), Spring Boot, and System Design.

Target Roles: Java Developer, Backend Engineer, Tech Lead | **Topics:** Core JVM, Concurrency, Spring Boot, Performance

1. Core Java & JVM Architecture

1.1 JVM Memory Model

Understanding memory management is vital for technical rounds. The JVM memory is structured into:

- **Heap Memory:** Shared across all threads; holds all instantiated objects and JRE classes. Divided into Young Generation (Eden, S0, S1) and Old (Tenured) Generation.
- **Stack Memory:** Dedicated per thread; stores local primitive variables, method execution frames, and references to objects in Heap.
- **Metaspace (Java 8+):** Replaced PermGen. Resides in native memory; stores class definitions, method metadata, and constant pool. Dynamically resizes automatically.

Key Interview Distinction (Stack vs. Heap): Stack allocations are fast and cleaned up automatically via LIFO when method calls return. Heap allocations require Garbage Collection (GC) overhead and throw `OutOfMemoryError`: Java heap space when exhausted.

1.2 Object Immutability & String Interning

To create a truly immutable class in Java:

1. Declare the class as `final` (prevents subclassing).
2. Make all fields `private` and `final`.
3. Do not provide setter methods.
4. Perform deep copy / defensive copying for mutable references in constructors and getters.

```
// Example of a Robust Immutable Class
public final class ImmutableUser {
    private final String id;
    private final List<String> roles;

    public ImmutableUser(String id, List<String> roles) {
        this.id = id;
        this.roles = (roles != null) ? List.copyOf(roles) : List.of();
    }

    public String getId() { return id; }
    public List<String> getRoles() { return roles; } // Safe: unmodifiable
}
```

2. Java Collections Framework Deep Dive

2.1 HashMap Internal Mechanics (Key Interview Focus)

- **Buckets & Hashing:** An array of Node objects (Node<K,V>[] table). Key index is calculated using $(n - 1) \& \text{hash}(\text{key})$.
- **Collision Resolution:** Handled via LinkedList. In Java 8+, if bucket length exceeds TREEIFY_THRESHOLD (8) and total table capacity ≥ 64 , the linked list transforms into a **Red-Black Tree**, optimizing lookup from $O(n)$ to $O(\log n)$.
- **Load Factor & Resizing:** Default capacity is 16, default load factor is 0.75. When size exceeds $16 * 0.75 = 12$, capacity doubles (32).

Collections Summary:

- **ArrayList:** Insertion Order, Null allowed, Not Thread-safe. $O(1)$ by index, $O(n)$ by value.
- **LinkedList:** Insertion Order, Null allowed, Not Thread-safe. $O(n)$ lookup.
- **HashMap:** Unordered, 1 Null Key & Multi Null Values, Not Thread-safe. $O(1)$ avg, $O(\log n)$ worst.
- **ConcurrentHashMap:** Unordered, No Null Keys/Values, Thread-safe (CAS + synchronized buckets). $O(1)$ avg.
- **TreeSet / TreeMap:** Sorted (Comparable/Comparator), No Null Keys, Not Thread-safe. $O(\log n)$ lookup.

3. Multithreading & Concurrency

3.1 Volatile, Synchronized, and Atomic Variables

- **volatile:** Guarantees *visibility* of changes across CPU caches directly into main memory; prevents instruction reordering. Does **not** guarantee atomicity (e.g. `count++` is not thread-safe with volatile alone).
- **synchronized:** Provides mutual exclusion and memory visibility via intrinsic monitor locks.
- **java.util.concurrent.atomic:** Uses CPU-level **Compare-And-Swap (CAS)** for lock-free, atomic operations with minimal overhead.

3.2 Executor Framework & ThreadPool Lifecycle

```
ThreadPoolExecutor executor = new ThreadPoolExecutor(  
    4, // corePoolSize  
    10, // maximumPoolSize  
    60L, TimeUnit.SECONDS, // keepAliveTime  
    new ArrayBlockingQueue<>(500), // bounded work queue  
    new ThreadPoolExecutor.CallerRunsPolicy() // rejection handler  
);
```

Java 21 Virtual Threads (Project Loom): Virtual threads are lightweight user-mode threads managed by the JVM rather than the OS kernel. They drastically reduce the memory footprint per thread (~few KB vs 1MB OS thread), enabling millions of concurrent I/O-bound operations without complex reactive programming.

4. Modern Java (8 to 21) Highlights

4.1 Streams API & Collectors

```
// Grouping employees by department and calculating average salary
Map<String, Double> avgSalaryByDept = employees.stream()
    .filter(e -> e.getStatus() == Status.ACTIVE)
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.averagingDouble(Employee::getSalary)
    ));
```

4.2 Records, Sealed Classes & Pattern Matching

- **Records (Java 16+)**: Concise immutable data carriers generating constructor, getters, equals, hashCode, and toString automatically.

- **Pattern Matching for Switch (Java 21)**: Type-safe conditional matching without verbose casting.

```
public sealed interface Shape permits Circle, Rectangle {}
public record Circle(double radius) implements Shape {}
public record Rectangle(double width, double height) implements Shape {}

public static double calculateArea(Shape shape) {
    return switch (shape) {
        case Circle c -> Math.PI * c.radius() * c.radius();
        case Rectangle r -> r.width() * r.height();
    };
}
```

5. Spring Boot & Microservices Core Concepts

SPRING CORE: What is Bean Scope and how does @Scope("prototype") behave inside @Scope("singleton")?

By default, Spring beans are singletons. If a prototype bean is injected into a singleton bean via standard @Autowired, it is initialized only once during startup. To get a fresh prototype instance each time, use @Lookup method injection or ObjectProvider<T>.

TRANSACTIONS: How does @Transactional handle rollback?

By default, Spring rolls back transactions only on unchecked exceptions (subclasses of RuntimeException and Error). Checked exceptions (Exception) do not trigger rollback unless explicitly configured with @Transactional(rollbackFor = Exception.class).

JPA / HIBERNATE: What is the N+1 Query Problem and how to resolve it?

The N+1 problem occurs when fetching a parent entity executes 1 query, and accessing its lazy-loaded child collection executes N individual queries. Solutions: JOIN FETCH in JPQL, @EntityGraph, or batch size configuration (hibernate.default_batch_fetch_size).

6. Frequently Asked Tricky Technical Questions

Q1: Why must equals() and hashCode() be overridden together?

The contract states: If two objects are equal according to equals(), their hashCode() must produce the exact same integer value. Failing to adhere to this contract breaks hash-based collections like HashMap, HashSet, leading to lost entries or duplicate insertions.

Q2: What is the difference between fail-fast and fail-safe iterators?

Fail-fast: Iterators (e.g. ArrayList.iterator()) throw ConcurrentModificationException if collection is modified structurally after iterator creation.

Fail-safe: Iterators (e.g. CopyOnWriteArrayList, ConcurrentHashMap) iterate over a clone or snapshot without throwing exceptions.

Q3: What is the Difference Between CompletableFuture and traditional Future?

Future requires blocking calls (.get()) to retrieve results.

CompletableFuture supports non-blocking asynchronous pipelines, chaining callbacks (thenApply, thenCompose), combining multiple futures (allOf, anyOf), and manual completion.